

Surface-primitive cell construction

(description of the algorithm implemented in CALM slab ops)

Reviewed by: CALM code reviewer

June 6, 2026

Abstract. This note documents and explains the algorithm implemented in `calm/slab/ops/primitive_surface_algorithm.py` for constructing a motif-preserving surface supercell for a given Miller index (h, k, l) . The algorithm constructs a primitive in-plane surface net when the integer basis satisfies $u \times v = \pm m$ for a primitive Miller vector m . It constructs a primitive 3D oriented cell before explicit layer repetition when $|m^T w| = 1$. With `layers > 1`, the returned slab is intentionally an integer supercell. All claims are conditional on the validation checks passing; otherwise the implementation raises rather than silently returning an incorrect cell.

Contents

1. Notation and preliminaries
2. High-level algorithm outline
3. Mathematical steps and equations
 - Transform between conventional and primitive cells
 - Miller-index transform
 - Exact Bezout / extended-gcd construction of the in-plane lattice kernel
 - Determination of stacking vector and shear minimization
 - 2D basis reduction (Gauss / Niggli)
 - Hermite Normal Form (HNF) for canonicalization of 2×2 relations
 - Rotation of the oriented slab so the surface normal is $+\hat{z}$
4. Implementation notes and edge cases

1 Notation and preliminaries

- Column convention for lattice matrices: the lattice matrix A is a 3×3 matrix whose columns are the direct-space basis vectors in Cartesian coordinates. Thus a lattice vector with fractional coordinates $f \in \mathbb{R}^3$ has Cartesian coordinates $r = A f$.

- Conventional cell: A_{conv} . - Primitive bulk cell: A_{prim} . - Miller index (given w.r.t. conventional cell): $h_{\text{conv}} = (h, k, l)^T$. - Integer transforms are (where possible) unimodular matrices U with $\det U = \pm 1$. These preserve the lattice without introducing fractional translations.

Important identity used in the code:

$$A_{\text{conv}} = A_{\text{prim}} P,$$

where P is the integer (in general rational) matrix giving the change of basis from primitive to conventional coordinates (the code computes P numerically via linear solves).

Reciprocal-lattice matrix corresponding to A (column convention) is

$$B = 2\pi (A^T)^{-1},$$

and a plane with Miller index m (in the same coordinate basis) has Cartesian normal vector proportional to Bm .

2 High-level algorithm outline (as implemented)

The implemented procedure (function `compute_primitive_surface_basis` and wrapper `build_oriented_primitive_slab`) follows these main steps:

1. Standardize / obtain primitive bulk cell A_{prim} and conventional cell A_{conv} ; form the transform P with $A_{\text{conv}} = A_{\text{prim}}P$.
2. Transform Miller index to primitive coordinates:

$$h_{\text{prim}} = P^{-T} h_{\text{conv}}.$$

Rationalize to obtain an integer vector m (reduce by gcd).

3. Construct integer vectors u , v , and w_B directly from the primitive Miller vector $m = (a, b, c)$ using extended-gcd / Bezout identities. The construction returns u and v in the integer kernel of m^T and a Bezout stacking vector w_B satisfying

$$m^T u = 0, \quad m^T v = 0, \quad u \times v = \pm m, \quad m^T w_B = 1.$$

4. Reduce the 2D in-plane basis (u, v) (Gauss reduction using their Cartesian lengths under A_{prim}) to get a short, stable in-plane basis. After any 2D reduction, the implementation ensures that the transform is integer-unimodular (determinant ± 1) and rechecks the primitive-kernel certificate $u \times v = \pm m$.
5. Choose the Bezout stacking vector w_B from the exact construction above, so that $|m^T w_B| = 1$ for primitive m . Minimize shear by considering integer-adjusted vectors $w_B + pu + qv$ and choosing the pair (p, q) that minimizes the 3D lattice-norm $\|A_{\text{prim}}(w_B + pu + qv)\|_2$ via least-squares + local integer search.
6. Form the integer transform matrix (column-wise) $T = [u \mid v \mid \text{layers} \cdot w]$ and construct the supercell of the primitive cell using T (use the project's ASE adapter which accepts column-basis matrices).
7. Compute the oriented cell $A_{\text{oriented}} = A_{\text{prim}}T$ and the oriented reciprocal $B_{\text{oriented}} = 2\pi(A_{\text{oriented}}^T)^{-1}$. The Miller vector in the oriented basis is

$$m_{\text{oriented}} = T^T m,$$

and the Cartesian normal must be computed as

$$g = B_{\text{oriented}} m_{\text{oriented}}.$$

Rotate the oriented cell so g becomes $+\hat{z}$ (Rodrigues rotation).

8. Apply a 2D Niggli (or Gauss) reduction to the in-plane lattice to obtain a canonical in-plane primitive cell (optionally apply integer unimodular $U_{2 \times 2}$).

3 Mathematical details and equations

3.1 Worked example: simple cubic (110)

Take primitive Miller vector $m = (1, 1, 0)^\top$. Then

$$g = \gcd(1, 1) = 1,$$

and one valid choice of Bezout coefficients is $p = 1$, $q = 0$, together with $s = 1$, $t = 0$ satisfying

$$sg + tc = 1.$$

The exact construction gives

$$u = (1, -1, 0), \quad v = (0, 0, 1), \quad w_B = (1, 0, 0).$$

The certificates are immediate:

$$u \times v = -(1, 1, 0) = -m, \quad m^\top w_B = 1.$$

Any shear-minimized stacking vector differs from w_B by an integer combination of u and v , so the determinant and kernel certificates remain unchanged.

3.2 Transform between conventional and primitive cells

We have

$$A_{\text{conv}} = A_{\text{prim}} P \Rightarrow P = A_{\text{prim}}^{-1} A_{\text{conv}}, \quad P^{-1} = A_{\text{conv}}^{-1} A_{\text{prim}}.$$

This P is used to move indices (Miller indices or integer lattice vectors) between the two bases.

3.3 Miller-index transform (conventional $\rightarrow$ primitive)

Given Miller index in conventional basis h_{conv} , the equivalent Miller index in primitive basis is

$$h_{\text{prim}} = P^{-T} h_{\text{conv}}.$$

Numerically this is computed by forming

$$h_{\text{prim}}^{(\text{float})} = P^{-T} h_{\text{conv}},$$

then rationalizing (see below) to obtain an integer triple which is finally reduced by gcd to produce a primitive integer Miller vector m .

3.4 Rationalization (float $\rightarrow$ integer)

A floating vector with small-denominator rational components is converted to integers by finding approximating rationals via continued fractions / limit denominator (implemented using Python's `Fraction.limit_denominator`), then rescaling by the least common multiple (LCM) of denominators:

$$h_{\text{prim}}^{(\text{float})} \approx \frac{1}{D} n, \quad n \in \mathbb{Z}^3, \quad D = \text{lcm}(\text{denominators}),$$

and finally reduce n by gcd to get the minimal integer triple m .

3.5 Exact Bezout construction of the primitive in-plane kernel

Let the primitive Miller vector be $m = (a, b, c)^\top$ with $\gcd(a, b, c) = 1$.

If $a = b = 0$, then necessarily $c = \pm 1$. A valid exact choice is

$$u = (1, 0, 0), \quad v = (0, c, 0), \quad w_B = (0, 0, c),$$

which gives

$$m^\top u = 0, \quad m^\top v = 0, \quad u \times v = m, \quad m^\top w_B = 1.$$

Otherwise let

$$g = \gcd(a, b),$$

and choose Bezout coefficients $p, q \in \mathbb{Z}$ such that

$$pa + qb = g.$$

Then choose integers s, t such that

$$sg + tc = 1.$$

Define

$$u = \left(\frac{b}{g}, -\frac{a}{g}, 0 \right), \quad v = (-cp, -cq, g), \quad w_B = (sp, sq, t).$$

Then the exact certificates are

$$m^\top u = 0, \quad m^\top v = 0, \quad u \times v = -m, \quad m^\top w_B = 1.$$

Therefore the in-plane basis is primitive and the oriented 3D cell before layer multiplication is primitive as well, since

$$\det[u \ v \ w_B] = (u \times v)^\top w_B = \pm 1.$$

3.6 2D (in-plane) reduction (Gauss / Niggli)

Once integer in-plane basis vectors u, v are available (in primitive fractional coordinates), the algorithm computes their Cartesian images under A_{prim} :

$$u_{\text{cart}} = A_{\text{prim}} u, \quad v_{\text{cart}} = A_{\text{prim}} v,$$

and then performs an iterative Gauss-type reduction to obtain a short, near-orthogonal basis. The key reduction step is the classical Gauss reduction coefficient

$$\mu = \frac{u_{\text{cart}} \cdot v_{\text{cart}}}{\|u_{\text{cart}}\|^2},$$

and replace

$$v \leftarrow v - \text{round}(\mu) u$$

repeating (and swapping $u \leftrightarrow v$ when needed) until $|\mu| \leq 1/2$ (within tolerance). The code also optionally applies a 2D Niggli reduction (a canonical metric-space reduction) to produce a canonical representative with a small unimodular integer transform $U_{2 \times 2}$.

3.7 Stacking vector w and shear minimization

The algorithm chooses the stacking vector w_B from the exact extended-gcd / Bezout construction above so that

$$m^\top w_B = 1.$$

For primitive m this implies the 3D oriented cell spanned by $[u \ v \ w_B]$ is itself primitive (determinant ± 1) before any explicit layer repetition. Note: this differs from a naive choice $w = m$, which generally gives $|m^\top m| > 1$ and hence a nonprimitive 3D supercell.

Since adding any integer linear combination of the in-plane lattice vectors leaves the stacking direction in the same lattice coset, the algorithm considers integer-adjusted stacking vectors

$$w(p, q) = w_B + pu + qv, \quad p, q \in \mathbb{Z},$$

and chooses (p^*, q^*) to minimize the metric objective

$$(p^*, q^*) = \operatorname{argmin}_{p, q \in \mathbb{Z}} \|A_{\text{prim}}(w_B + pu + qv)\|_2.$$

Because $A_{\text{prim}}u$ and $A_{\text{prim}}v$ lie in the surface plane and adding $pu + qv$ does not change the normal component, this is equivalent to minimizing the in-plane component of the stacking vector in the lattice metric. The implementation solves the continuous least-squares problem in Cartesian space to find real (p, q) , rounds to the nearest integers, then performs a small local integer search around the rounded solution (e.g. ± 2) and selects the integer pair that minimizes the metric norm. The final stacking vector is then

$$w = w_B + p^*u + q^*v.$$

This procedure is rotation-invariant up to exact ties.

3.8 Hermite Normal Form (HNF) for canonical supercell relations (2×2)

When mapping between the conventional (possibly centered) surface lattice and the primitive surface lattice, the code sometimes computes a small integer 2×2 matrix M describing how one set of in-plane integer coordinates maps to another. To canonicalize this relation the Hermite Normal Form (HNF) of M is used: find unimodular integer W and upper-triangular integer H with non-negative diagonal such that

$$M = W H,$$

and $0 \leq H_{0,1} < H_{1,1}$ for a 2×2 HNF. HNF gives a canonical representative of integer sublattices and is used to choose standard supercell transforms.

3.9 Forming the integer transform T and building the oriented block

After u, v, w are determined (all integer vectors expressed in primitive fractional coordinates), the 3×3 integer transformation used to make the oriented block is

$$T = [u \ v \ \text{layers} \cdot w]$$

(as columns). The ASE-based supercell is created from the primitive cell via this integer column transform. The oriented lattice becomes

$$A_{\text{oriented}} = A_{\text{prim}} T.$$

The (absolute) determinant $|\det T|$ equals the integer number of primitive cells in the supercell (a check on layer count and periodic multiplicity).

3.10 Surface normal and rotation to slab frame

Given A_{oriented} and the integer Miller vector m (in the original primitive basis), the Cartesian surface normal must be computed consistently. After forming $A_{\text{oriented}} = A_{\text{prim}} T$ the Miller vector in the oriented basis is

$$m_{\text{oriented}} = T^{\text{T}} m.$$

Compute the oriented reciprocal and the true Cartesian normal using the oriented Miller:

$$B_{\text{oriented}} = 2\pi (A_{\text{oriented}}^{\text{T}})^{-1}, \quad g = B_{\text{oriented}} m_{\text{oriented}}.$$

Equivalently, the normal may be computed in the primitive frame before changing basis as

$$g = B_{\text{prim}} m.$$

Normalize

$$\hat{g} = g / \|g\|.$$

The target unit vector is $\hat{z} = (0, 0, 1)^{\text{T}}$. The rotation R that sends $\hat{g} \mapsto \hat{z}$ is built with the Rodrigues formula (special-case handling if $\hat{g} = \pm \hat{z}$):

$$v = \hat{g} \times \hat{z}, \quad s = \|v\|, \quad c = \hat{g} \cdot \hat{z},$$

$$R = I + [v]_{\times} + \frac{[v]_{\times}^2}{s^2}(1 - c),$$

where $[v]_{\times}$ is the skew symmetric cross-product matrix of v . Apply R to cell and positions; after rotation the first two lattice vectors should have negligible z -components. The implementation only zeroes third-axis x/y components when they are below a small numerical tolerance; if non-negligible shear remains, it emits a warning and leaves the geometry unchanged.

4 Implementation notes and edge cases

- The code always constructs the surface cell from the primitive bulk cell, which avoids using a nonprimitive bulk lattice and thereby avoids fractional motif translations at the bulk level.
- The in-plane surface net is primitive when the integer basis satisfies $u \times v = \pm m$ for a primitive Miller vector m .
- The full 3D oriented cell *before* explicit layer multiplication is primitive only when $|m^{\text{T}} w| = 1$ (which is enforced by choosing a Bezout stacking vector). With an explicit layer multiplier, the resulting slab cell is an integer supercell with determinant scaled by the number of layers.

- The primitive in-plane kernel and Bezout stacking vector are constructed exactly for every primitive Miller vector using extended-gcd identities. Pathological inputs such as the zero Miller vector are rejected explicitly.

- Rationalization uses a maximum denominator (default 12) to avoid spuriously large integer scalings coming from floating numerical noise. After forming integer numerators, vectors are reduced by their gcd to produce minimal integer Miller indices m .

- The in-plane reduction is metric-aware (it uses Cartesian lengths computed from A_{prim}) to produce shorter, numerically stable in-plane basis vectors. After this a 2D Niggli reduction may be applied for canonicalization; the integer unimodular transform returned by Niggli is rounded to integer and applied.
- The in-plane reduction is metric-aware (it uses Cartesian lengths computed from A_{prim}) to produce shorter, numerically stable in-plane basis vectors. After this a 2D Niggli

reduction may be applied for canonicalization; the integer unimodular transform returned by Niggli is rounded to integer and applied. The code rejects non-integer or non-unimodular transforms when they would change the lattice kernel.

- Shear minimization is solved as a real least-squares problem and its coefficients are rounded to integers. This is a heuristic for choosing a short representative in the integer coset $w_B + \mathbb{Z}u + \mathbb{Z}v$; it does not affect primitive-lattice correctness because adding $pu + qv$ preserves $m^T w = 1$ and the determinant certificates.

- HNF is implemented in a straightforward way for small 2×2 integer matrices. Care is taken to make diagonal entries non-negative and reduce off-diagonal entries to canonical range.

- Numerical tolerances: several checks use small epsilons (e.g., for nearly-colinear vectors) and rounding. The code documents the possibility that tiny numerical artifacts may change e.g. handedness; these are considered negligible for geometry construction but should be monitored when high exactness is required.

- The determinant of T is used as an integrity check (number of primitive cells). If det is not integer within tolerance, it indicates an upstream numerical issue.
- ASE convention: CALM uses a column-vector convention for lattice matrices (columns are basis vectors). ASE stores cell vectors as rows. The helper `calm.ase_adapter.supercell_matrix_ase_from_col(P_col)` returns P_col^T for passing into `ase.build.make_supercell`. The function `make_supercell_col(...)` is the single canonical adapter used in this codebase and accepts column-basis matrices.

5 Implementation excerpts used for validation

The snippets below are verbatim excerpts from the working tree after the validation edits. They include the exact code paths and function bodies used in the tests discussed above. Omitted lines (“...” inside snippets) denote non-essential boilerplate; each listing is labelled with its source.

Source file	Function / test	Purpose	Included
calm/slab/ops/primitive_surface_algorithm.py	compute_primitive_surface_basis	Miller transform, exact Bezout kernel, reductions, stacking, shear	excerpted (core blocks)
calm/slab/ops/primitive_surface_algorithm.py	minimize_shear	shear minimization (metric LS + local search)	full
calm/ase_adapter.py	make_supercell_col	ASE adapter (row/col convention)	excerpt
calm/slab/ops/validate_surface_basis.py	validate_surface_basis	integer and geometric invariants	full
tests/slab/test_surface_basis_validation.py	test cases	normal formula + determinant checks	excerpt
tests/slab/test_exhaustive_millers.py	exhaustive Miller tests	primitive-kernel coverage	full

1. Miller-index transform and rationalization

```
# Step 1: Compute transformation matrix P such that A_conv = A_prim @ P
P = np.linalg.solve(A_prim, A_conv) # A_conv = A_prim @ P
P_inv = np.linalg.solve(A_conv, A_prim) # A_prim = A_conv @ P_inv

# Step 2: Transform Miller indices to primitive coordinates
h_conv = np.array([h, k, l], dtype=float)
h_prim_float = P_inv.T @ h_conv
```

```

# Step 3: Rationalize to integers
numerators, denom = rationalize_vector(h_prim_float, max_denom=12)
g = gcd_multiple([abs(n) for n in numerators if n != 0])
m = np.array([n // g for n in numerators], dtype=int)

# Validate that the integer primitive Miller m is consistent with the
# original conventional Miller index under the basis transform.
v_test = P.T @ m
if not np.allclose(np.cross(v_test, h_conv), 0.0, atol=1e-6):
    raise ValueError("Rationalized primitive Miller vector 'm' is not
        parallel to h_conv")

```

1b. Exact rational Miller transform helper

```

def primitive_miller_from_conventional(A_conv, A_prim, h_conv, *,
                                     max_denominator=12, atol=1e-10,
                                     rtol=1e-10):

    P = np.linalg.solve(A_prim, A_conv)
    P_rat = _fraction_matrix_from_float(P, max_denominator)
    P_rat_float = np.array([[float(x) for x in row] for row in P_rat],
                           dtype=float)
    recon = A_prim @ P_rat_float
    scale = max(np.linalg.norm(A_conv), np.linalg.norm(recon), 1.0)
    if not np.allclose(recon, A_conv, atol=atol + rtol * scale, rtol=0.0):
        raise ValueError("Rationalized basis transform does not reproduce
            A_conv within tolerance")

    P_inv_rat = _fraction_inverse3(P_rat)
    P_inv_T_rat = [[P_inv_rat[j][i] for j in range(3)] for i in range(3)]
    h_frac = [Fraction(int(x), 1) for x in h_conv]
    h_prim_frac = _fraction_matvec(P_inv_T_rat, h_frac)
    denoms = [f.denominator for f in h_prim_frac]
    common_denom = lcm_multiple(denoms)
    numerators = [int(f.numerator * (common_denom // f.denominator)) for f
        in h_prim_frac]
    # Fraction-based inverse and exact rational Miller transform; if this
    # exact
    # rational path cannot be verified, the implementation raises rather
    # than
    # silently accepting an inconsistent transform.

```

2. Exact primitive-kernel construction returning u, v, w

```

def _primitive_surface_triplet_from_m(m):
    a, b, c = map(int, m.tolist())
    if a == 0 and b == 0:
        u = np.array([1, 0, 0], dtype=int)
        v = np.array([0, c, 0], dtype=int)
        w = np.array([0, 0, c], dtype=int)
    else:

```

```

    g, p, q = _extended_gcd_pair(a, b)
    g2, s, t = _extended_gcd_pair(g, c)
    if g2 != 1:
        raise ValueError("Primitive m must satisfy gcd(g, c)=1")
    u = np.array([b // g, -a // g, 0], dtype=int)
    v = np.array([-c * p, -c * q, g], dtype=int)
    w = np.array([s * p, s * q, t], dtype=int)

    cross_uv = np.cross(u, v)
    if not (np.array_equal(cross_uv, m) or np.array_equal(cross_uv, -m)):
        raise ValueError("Exact construction failed primitive certificate")
    if int(np.dot(m, w)) != 1:
        raise ValueError("Exact construction failed Bezout certificate")
    return u, v, w

u, v, w_B = _primitive_surface_triplet_from_m(m)

```

3. In-plane reduction and unimodular checks

```

def _reduce_2d_basis(u, v, A_cart, max_iter=100):
    u_curr = u.copy()
    v_curr = v.copy()
    for _ in range(max_iter):
        u_cart = A_cart @ u_curr
        v_cart = A_cart @ v_curr
        u_norm_sq = np.dot(u_cart, u_cart)
        v_norm_sq = np.dot(v_cart, v_cart)
        if v_norm_sq < u_norm_sq:
            u_curr, v_curr = v_curr, u_curr
            u_norm_sq, v_norm_sq = v_norm_sq, u_norm_sq
            u_cart, v_cart = v_cart, u_cart
        uv_dot = np.dot(u_cart, v_cart)
        if abs(u_norm_sq) < 1e-10:
            break
        mu = uv_dot / u_norm_sq
        if abs(mu) <= 0.5 + 1e-6:
            break
        k = int(round(mu))
        v_curr = v_curr - k * u_curr
    return u_curr, v_curr

u, v = _reduce_2d_basis(u, v, A_prim, max_iter=100)

def _final_reduce(vec):
    vec = np.asarray(vec, dtype=int)
    g = gcd_multiple([abs(x) for x in vec if x != 0])
    return vec // g if g > 0 else vec

u = _final_reduce(u)
v = _final_reduce(v)

def apply_2d_niggli_reduction(slab):

```

```

_, U2, _ = niggli_reduce_2d(S2, tol=1e-5)
U2i = np rint(U2).astype(int)
det_u2 = int(round(np.linalg.det(U2i)))
if not np.allclose(U2, U2i, atol=1e-8) or abs(det_u2) != 1:
    warnings.warn("Rounded 2D Niggli transform is not exactly integer-
        unimodular; falling back to original basis.")
    return slab.copy(), np.eye(2, dtype=int)
U3 = np.eye(3, dtype=int)
U3[:2, :2] = U2i
cell_new = slab.cell.array @ U3.T
positions_frac = slab.get_scaled_positions()
U3_inv = np.linalg.inv(U3)
positions_frac_new = (U3_inv @ positions_frac.T).T
slab_reduced = slab.copy()
slab_reduced.set_cell(cell_new, scale_atoms=False)
slab_reduced.set_scaled_positions(positions_frac_new)
slab_reduced.wrap(eps=1e-5)
return slab_reduced, U2i

```

4. Stacking-vector selection and shear minimization

```

u, v, w_B = _primitive_surface_triplet_from_m(m)

w = w_B.copy()

# minimize shear via metric-aware least squares + local integer search
def minimize_shear(w, u, v, A_prim):
    w_cart = A_prim @ w
    u_cart = A_prim @ u
    v_cart = A_prim @ v
    M = np.column_stack([u_cart, v_cart])
    coeffs = np.linalg.lstsq(M, -w_cart, rcond=None)[0]
    p, q = np.round(coeffs).astype(int)
    best = None
    best_norm = None
    for dp in range(-2, 3):
        for dq in range(-2, 3):
            w_try = w + int(p+dp)*u + int(q+dq)*v
            norm_try = np.linalg.norm(A_prim @ w_try)
            if best is None or norm_try < best_norm:
                best = w_try
                best_norm = norm_try
    return best

w = minimize_shear(w, u, v, A_prim)

```

5. Construction of the integer transform T and ASE call

```

T = np.column_stack([u, v, layers * w]).astype(int)

# Use CALM ASE adapter (converts column P_col to ASE row matrix P_row =
    P_col.T)

```

```
from calm.ase_adapter import make_supercell_col
slab = make_supercell_col(atoms_prim, T)
```

6. ASE adapter (excerpt)

```
def supercell_matrix_ase_from_col(P_col: np.ndarray) -> np.ndarray:
    P = np.asarray(P_col, dtype=int)
    return P.T

def make_supercell_col(atoms: "Atoms", P_col: np.ndarray, *, wrap: bool =
    True) -> "Atoms":
    from ase.build import make_supercell as _ase_make_supercell
    P_row = supercell_matrix_ase_from_col(P_col)
    try:
        return _ase_make_supercell(atoms, P_row, wrap=wrap)
    except TypeError:
        return _ase_make_supercell(atoms, P_row)
```

7. Reciprocal normal computation and rotation

```
B_oriented = 2 * np.pi * np.linalg.inv(A_oriented.T)
m_oriented = T.T @ m
g_cart = B_oriented @ m_oriented
g_hat = g_cart / np.linalg.norm(g_cart)

z_hat = np.array([0, 0, 1])
if np.allclose(g_hat, z_hat):
    R = np.eye(3)
elif np.allclose(g_hat, -z_hat):
    R = np.diag([1.0, -1.0, -1.0])
else:
    v = np.cross(g_hat, z_hat)
    s = np.linalg.norm(v)
    c = np.dot(g_hat, z_hat)
    vx = np.array([[0, -v[2], v[1]], [v[2], 0, -v[0]], [-v[1], v[0], 0]])
    R = np.eye(3) + vx + (vx @ vx) * ((1 - c) / (s * s))

# Apply rotation to slab cell and positions
slab.set_cell(slab.cell.array @ R.T, scale_atoms=False)
slab.set_positions(slab.get_positions() @ R.T)

# Tolerance-based zeroing of tiny third-axis xy components
cell_new = slab.cell.array.copy()
if abs(cell_new[2,0]) <= 1e-8 and abs(cell_new[2,1]) <= 1e-8:
    c_z = cell_new[2,2]
    cell_new[2] = np.array([0.0, 0.0, c_z])
    slab.set_cell(cell_new, scale_atoms=False)
else:
    warnings.warn("Third lattice vector has non-negligible xy components;
        not zeroing")
```

8. Validation helper (full)

```
def validate_surface_basis(A_prim: np.ndarray, m: np.ndarray, u: np.
    ndarray,
                        v: np.ndarray, w: np.ndarray, layers: int = 1,
                        atol: float = 1e-8) -> Tuple[bool, dict]:
    """Validate integer and geometric invariants for a surface basis."""

    info = {}

    m = np.asarray(m, dtype=int)
    u = np.asarray(u, dtype=int)
    v = np.asarray(v, dtype=int)
    w = np.asarray(w, dtype=int)

    info['m'] = m.tolist()
    g = np.gcd.reduce(np.abs(m))
    info['gcd_m'] = int(g)
    ok = True
    if g != 1:
        info['error_gcd_m'] = True
        ok = False

    info['m_dot_u'] = int(np.dot(m, u))
    info['m_dot_v'] = int(np.dot(m, v))
    info['m_dot_w'] = int(np.dot(m, w))
    if info['m_dot_u'] != 0 or info['m_dot_v'] != 0:
        info['error_kernel'] = True
        ok = False

    cross_uv = np.cross(u, v)
    info['cross_uv'] = cross_uv.tolist()
    primitive_ok = np.array_equal(cross_uv, m) or np.array_equal(cross_uv,
        -m)
    info['cross_coeff_q'] = 1 if np.array_equal(cross_uv, m) else (-1 if
        np.array_equal(cross_uv, -m) else None)
    if not primitive_ok:
        info['error_primitivity'] = True
        ok = False

    det_T_signed = int(np.dot(np.cross(u, v), layers * w))
    expected_signed = int(layers * np.dot(m, w))
    info['det_T_signed'] = det_T_signed
    info['expected_det_signed'] = expected_signed
    info['det_T'] = abs(det_T_signed)
    info['expected_det'] = abs(expected_signed)
    if abs(det_T_signed) != abs(expected_signed):
        info['error_det_mismatch'] = True
        ok = False

    A_prim = np.asarray(A_prim, dtype=float)
    a_s = A_prim @ u
    b_s = A_prim @ v
    B_prim = 2 * np.pi * np.linalg.inv(A_prim.T)
```

```

g_cart = B_prim @ m
info['dot_g_as'] = float(np.dot(g_cart, a_s))
info['dot_g_bs'] = float(np.dot(g_cart, b_s))
if not (abs(info['dot_g_as']) < atol and abs(info['dot_g_bs']) < atol):
    info['error_geometric_orthogonality'] = True
    ok = False

area_lhs = np.linalg.norm(np.cross(a_s, b_s))
area_rhs = abs(np.linalg.det(A_prim)) * np.linalg.norm(np.linalg.solve(
    (A_prim.T, m)))
info['area_lhs'] = float(area_lhs)
info['area_rhs'] = float(area_rhs)
if not (abs(area_lhs - area_rhs) < max(atol, 1e-8 * area_rhs)):
    info['error_area_mismatch'] = True
    ok = False

return ok, info

```

9. Representative tests added

```

def test_cubic_110_normal_formula_and_determinant():
    u, v, w, m = compute_primitive_surface_basis(1, 1, 0, A_conv, A_prim)
    T = np.column_stack([u, v, w])
    A_oriented = A_prim @ T
    B_prim = 2*pi*inv(A_prim.T)
    B_oriented = 2*pi*inv(A_oriented.T)
    m_oriented = T.T @ m
    assert np.linalg.norm(np.cross(B_prim @ m, B_oriented @ m_oriented)) <
        1e-8

def test_rotation_special_cases_and_handedness():
    slab_plus, R_plus = build_oriented_primitive_slab(atoms_plus, 0, 0, 1,
        A_conv, layers=1)
    assert np.allclose(R_plus.T @ R_plus, np.eye(3))
    assert np.isclose(np.linalg.det(R_plus), 1.0)
    slab_minus, R_minus = build_oriented_primitive_slab(atoms_minus, 0, 0,
        -1, A_conv, layers=1)
    assert np.allclose(R_minus.T @ R_minus, np.eye(3))
    assert np.isclose(np.linalg.det(R_minus), 1.0)

def test_fcc_conventional_to_primitive_millers():
    A_conv = np.eye(3)
    A_prim = np.array([[0.0, 0.5, 0.5], [0.5, 0.0, 0.5], [0.5, 0.5, 0.0]])
    cases = {(1,0,0): np.array([0,1,1]), (1,1,0): np.array([1,1,2]),
        (1,1,1): np.array([1,1,1])}
    P = np.linalg.solve(A_prim, A_conv)
    for hkl, expected in cases.items():
        u, v, w, m = compute_primitive_surface_basis(*hkl, A_conv, A_prim)
        assert np.gcd.reduce(np.abs(m)) == 1
        assert np.linalg.norm(np.cross(P.T @ m, np.array(hkl, dtype=float)
            )) < 1e-8

```

```

        assert np.linalg.norm(np.cross(m.astype(float), expected.astype(
            float)))) < 1e-8
    ok, info = validate_surface_basis(A_prim, m, u, v, w, layers=1)
    assert ok, info

def test_bcc_conventional_to_primitive_millers():
    A_conv = np.eye(3)
    A_prim = np.array([[0.5, 0.5, -0.5], [0.5, -0.5, 0.5], [-0.5, 0.5,
        0.5]])
    cases = {(1,0,0): np.array([1,1,-1]), (1,1,0): np.array([1,0,0]),
        (1,1,1): np.array([1,1,1])}
    P = np.linalg.solve(A_prim, A_conv)
    for hkl, expected in cases.items():
        u, v, w, m = compute_primitive_surface_basis(*hkl, A_conv, A_prim)
        assert np.gcd.reduce(np.abs(m)) == 1
        assert np.linalg.norm(np.cross(P.T @ m, np.array(hkl, dtype=float)
            )) < 1e-8
        assert np.linalg.norm(np.cross(m.astype(float), expected.astype(
            float)))) < 1e-8
        ok, info = validate_surface_basis(A_prim, m, u, v, w, layers=1)
        assert ok, info

def test_motif_position_preservation_diamond():
    atoms = diamond_primitive_cell()
    A_prim = atoms.cell.array.T
    u, v, w, m = compute_primitive_surface_basis(1, 1, 0, A_prim, A_prim)
    T = np.column_stack([u, v, 2 * w]).astype(int)
    supercell = make_supercell_col(atoms, T)
    motif_frac = atoms.get_scaled_positions() % 1.0
    super_frac_in_prim = np.linalg.solve(A_prim, supercell.get_positions()
        .T).T % 1.0
    counts = np.zeros(len(motif_frac), dtype=int)
    for f in super_frac_in_prim:
        diffs = motif_frac - f
        diffs -= np.round(diffs)
        norms = np.linalg.norm(diffs, axis=1)
        idx = int(np.argmin(norms))
        assert norms[idx] < tol
        counts[idx] += 1
    assert np.all(counts == int(round(abs(np.linalg.det(T)))))

def test_motif_position_preservation_rocksalt_species_aware():
    atoms = rocksalt_primitive_cell()
    u, v, w, m = compute_primitive_surface_basis(1, 1, 1, A_conv, A_prim)
    T = np.column_stack([u, v, 2 * w]).astype(int)
    _assert_species_preserving_supercell(atoms, T)

def test_motif_position_preservation_custom_low_symmetry_species_aware():
    atoms = custom_low_symmetry_motif_cell()
    u, v, w, m = compute_primitive_surface_basis(2, 1, 3, A_conv, A_prim,
        max_denominator=256)
    T = np.column_stack([u, v, 2 * w]).astype(int)
    _assert_species_preserving_supercell(atoms, T)

```

```

def test_exhaustive_small_millers():
    for h,k,l in small_range:
        u,v,w,m = compute_primitive_surface_basis(h,k,l,...)
        ok, info = validate_surface_basis(...)
        assert ok

def test_large_miller_regression_cases():
    for h, k, l in [(7,5,3), (11,7,5), (13,8,5), (17,11,6), (19,13,7),
(29,17,11), (-23,14,9)]:
        u, v, w, m = compute_primitive_surface_basis(h, k, l, A_conv,
            A_prim)
        assert np.gcd.reduce(np.abs(m)) == 1
        assert int(np.dot(m, u)) == 0
        assert int(np.dot(m, v)) == 0
        cross_uv = np.cross(u, v)
        assert np.array_equal(cross_uv, m) or np.array_equal(cross_uv, -m)
        assert abs(int(np.dot(m, w))) == 1
        ok, info = validate_surface_basis(A_prim, m, u, v, w, layers=1)
        assert ok, info

def test_negative_miller_pairs_equivalent():
    for pos, neg in [((1,1,0), (-1,-1,0)), ((1,1,1), (-1,-1,-1)), ((2,1,3),
(-2,-1,-3))]:
        assert abs(_surface_area(A_prim, u1, v1) - _surface_area(A_prim,
            u2, v2)) < 1e-8
        _, R1 = build_oriented_primitive_slab(atoms, *pos, A_conv, layers
            =1)
        _, R2 = build_oriented_primitive_slab(atoms, *neg, A_conv, layers
            =1)
        assert np.isclose(np.linalg.det(R1), 1.0, atol=1e-8)
        assert np.isclose(np.linalg.det(R2), 1.0, atol=1e-8)

def test_primitive_basis_change_invariance():
    S = np.array([[1, 1, 0], [0, 1, 0], [0, 0, 1]], dtype=int)
    A_prim_new = A_prim @ S
    m_new = S.T @ m0
    u_new, v_new, w_new = _primitive_surface_triplet_from_m(m_new)
    u_old_coords = S @ u_new
    v_old_coords = S @ v_new
    cross_old = np.cross(u_old_coords, v_old_coords)
    assert np.array_equal(cross_old, int(round(np.linalg.det(S))) * m0) or
        np.array_equal(cross_old, -int(round(np.linalg.det(S))) * m0)

def test_failure_modes():
    with pytest.raises(ValueError, match="Miller indices cannot all be
        zero"):
        primitive_miller_from_conventional(A, A, (0, 0, 0))
    ...

def test_validate_surface_basis_scale_aware_small_and_large_cells():
    for a in [1e-4, 1.0, 1e4]:
        A_prim = simple_cubic_cell(a)
        u, v, w, m = compute_primitive_surface_basis(2, 1, 3, A_prim,
            A_prim)

```

```

ok, info = validate_surface_basis(A_prim, m, u, v, w, layers=3,
    atol=1e-8, rtol=1e-10)
assert ok, info

```

6 Summary of main equations

Below are the core relations used by the algorithm (notation as above):

$$A_{\text{conv}} = A_{\text{prim}} P$$

$$h_{\text{prim}} = P^{-T} h_{\text{conv}} \quad (\text{then rationalize to integer } m)$$

The exact constructive certificates are

$$m^T u = 0, \quad m^T v = 0, \quad u \times v = \pm m, \quad m^T w_B = 1.$$

Consequently,

$$\det[u \ v \ w_B] = (u \times v)^T w_B = \pm 1.$$

In Cartesian coordinates

$$u_{\text{cart}} = A_{\text{prim}} u, \quad v_{\text{cart}} = A_{\text{prim}} v.$$

Gauss reduction uses the coefficient

$$\mu = \frac{u_{\text{cart}} \cdot v_{\text{cart}}}{\|u_{\text{cart}}\|^2}$$

and replaces $v \leftarrow v - \text{round}(\mu) u$ iteratively to shorten the basis. After any 2D reduction (Gauss or Niggli), the implementation verifies that the 2×2 transform is integer-unimodular ($\det = \pm 1$) and rechecks the primitive-kernel certificate $u \times v = \pm m$.

The stacking vector is chosen as the exact Bezout vector w_B with $|m^T w_B| = 1$, and integer shear adjustments are applied via the metric objective described above.

The integer column transform used to build the oriented block is

$$T = [u \mid v \mid \text{layers} \cdot w],$$

and the oriented lattice is

$$A_{\text{oriented}} = A_{\text{prim}} T.$$

The Miller vector in the oriented basis is $m_{\text{oriented}} = T^T m$, and the Cartesian normal is computed as

$$B_{\text{oriented}} = 2\pi(A_{\text{oriented}}^T)^{-1}, \quad g = B_{\text{oriented}} m_{\text{oriented}}.$$

Finally apply Rodrigues rotation to map $\hat{g}$ to $\hat{z}$.

7 References to code

The functions implementing the above operations (in `calm/slab/ops/primitive_surface_algorithm.py`) include:

- `compute_primitive_surface_basis(...)` – main routine that produces the integer vectors u, v, w and the primitive Miller m .

- `_primitive_surface_triplet_from_m(...)` – exact extended-gcd construction of u , v , and w_B satisfying the primitive-kernel and Bezout certificates.
- `rationalize_vector(...)` – float-to-integer conversion using Python’s `Fraction` with an LCM rescaling.
- `_reduce_2d_basis(...)` – Gauss reduction using Cartesian norms.
- `minimize_shear(...)` – metric-aware least-squares + local integer search to minimize stacking vector shear.
- `_hermite_normal_form_2x2(...)` – small HNF implementation used to canonicalize 2×2 integer relations.
- `build_oriented_primitive_slab(...)` – wrapper that builds the oriented supercell and rotates it to slab frame; uses `calm.ase_adapter.make_supercell_col`.
- `apply_2d_niggli_reduction(...)` – calls the project’s 2D Niggli reducer and applies the integer unimodular transform when valid.

(See `calm/slab/ops/primitive_surface_algorithm.py` for the concrete code.)

8 Limitations and potential improvements

- The primitive in-plane kernel and Bezout stacking vector are constructed exactly for every primitive Miller vector using extended-gcd identities. Remaining exactness risks are primarily in rationalizing the conventional-to-primitive Miller transform and in any optional Niggli transform rounding. - Rationalization with a small denominator cap (default 12) is pragmatic but may fail for deliberately large-denominator transforms; increasing the cap or using exact arithmetic (rationals from the start) would be safer. - The shear minimization rounds real least-squares coefficients to integers; a direct integer optimization (short lattice vector search in 2D) could yield optimal shear-minimized w but at higher cost. - HNF for larger subproblems could be replaced by a robust integer-matrix library (e.g., via `sympy` or `sage`) if exact integer canonical forms are required. - After rotation, the first two lattice vectors should have negligible z -components. The implementation only zeroes third-axis x/y components when they are below a small numerical tolerance; if non-negligible shear remains, it emits a warning and leaves the geometry unchanged.

9 Conclusion

The implementation combines integer-lattice arithmetic (exact Bezout / extended-gcd construction, kernel certificates, determinant identities) with metric-aware Cartesian operations (Gauss reduction, shear minimization, rotation to slab frame) to produce a motif-preserving surface supercell from a primitive bulk cell. The exact invariants prove a primitive in-plane surface net when $u \times v = \pm m$ for primitive m . The oriented 3D cell before explicit layer multiplication is primitive when $|m^\top w| = 1$. With a layer multiplier, the slab is intentionally an integer supercell. Motif preservation means the output cell is generated by an integer transform from the primitive bulk cell and the motif positions are carried through without fractional translations. If the validation checks fail, the code raises rather than returning a silently incorrect surface cell.